机器学习零基础讲义

导论与回归

面向没有机器学习基础、Python 基础一般的自学者

根据《机器学习简介 · 第一讲》《机器学习导论 · 第 2 讲：回归》
及《回归：从条件期望到正则化》整理、核算并补充

个人学习使用 · 2026 年 9 月

这份讲义用来做什么

这是一份**个人自学和复习讲义**：把两讲课件中的概念、案例和回归方法串成可以独立阅读的路线，并用第三份 PDF 补足推导。它适合课前预习、课后复习、做题时查公式和第一次运行 Python 例子。它覆盖**机器学习导论与回归**，并不是整门机器学习课程的全部；分类、聚类、神经网络训练等后续内容在此只建立地图，不作完整课程展开。

建议用法：第一次读，依次读第 1–7 章和第 10 章，先理解句子、表格与手算；第 8–9 章的矩阵推导和优化可以第二遍再读；第 11 章代码边运行边改数字；最后做第 12 章自测。遇到公式先问：输入是什么、输出是什么、每个符号代表什么、它解决哪个问题，再看计算。

范围与来源：第一讲约 61 张幻灯片，第二讲约 62 张幻灯片；第三份 21 页 PDF 聚焦回归。本文用自己的例子和表述重新组织内容。课件中的案例和表格只用于教学演示，不能作为真实房价或产品性能证据。文末列出三份本地来源和对应章节。

目录

这份讲义用来做什么 **i**

第一章　先认识机器学习在做什么 **1**

1.1　一个不神秘的定义 1

1.2　统一的五步链条 1

1.3　三种学习范式 1

1.4　用课件案例读懂共同框架 2

1.5　常见分支怎样放进这张地图 2

第二章　读公式前的数学与 Python 预备 **3**

2.1　变量、函数、求和 3

2.2　向量和矩阵只需先会看形状 3

2.3　平均、方差和条件期望 3

2.4　导数与梯度的直观理解 3

2.5　Python 最小工具箱 4

第三章　从数据到可检验的学习问题 **5**

3.1　数据不是现实本身 5

3.2　训练误差与未来误差 5

3.3　训练、验证、测试三种用途 5

3.4　欠拟合、过拟合与偏差—方差 5

第四章　回归：连续数值预测 **7**

4.1　什么是回归 7

4.2　回归函数与不可约噪声 7

4.3　损失与评价指标 7

第五章　K 近邻回归：从“像谁”开始 **8**

5.1　算法与手算 8

5.2　为什么必须关注特征尺度 8

第六章　线性回归：从直线到矩阵 **9**

6.1　模型、截距、残差 9

6.2 最小二乘及其解 9
6.3 几何与数值计算 9
6.4 统计假设与解释边界 10

第七章 梯度下降：不会一步算出时怎样找答案 **11**

第八章 复杂模型、正则化与概率解释 **12**
8.1 多项式仍可用线性回归训练 12
8.2 Ridge、Lasso、Elastic Net 12
8.3 为什么高斯假设会导出平方损失 12
8.4 进阶选读：软阈值与 ISTA 13

第九章 完整的回归工作流 **14**

第十章 跟着敲两段 Python **15**
10.1 不用第三方包，手算误差 15
10.2 可重复的模拟数据练习 15

第十一章 自测题与参考答案 **17**

第十二章 一页公式索引与术语表 **18**

第十三章 资料对应与继续学习 **19**
13.1 这份讲义怎样对应原始资料 19
13.2 下一步学什么 19
13.3 软件资料 19

第一章　先认识机器学习在做什么

1.1　一个不神秘的定义

Tom Mitchell 的常用表述是：如果程序在任务 T 上的表现 P 随经验 E 增加而改善，就说它从经验中学习。比如识别垃圾邮件：T 是判断邮件类别，P 可以是误报率，E 是已经标注的邮件。三个字母能防止一句空泛的“模型更聪明了”掩盖评估问题。

传统程序通常由人写明规则；机器学习程序由人指定问题、数据表示、模型范围和评价方式，让算法从样本中估计参数。人仍然要负责**收集什么数据、标签是否可靠、犯哪种错更贵、部署时是否还能用**。

1.2　统一的五步链条

给定输入 x，模型 f_θ 产生预测 $\hat{y} = f_\theta(x)$。将预测与正确答案 y 比较，得到损失 $\ell(y, \hat{y})$；优化算法调整参数 θ；最后在未见过的数据上检验。简写为

$$\boxed{\text{数据} \longrightarrow \text{模型} \longrightarrow \text{损失} \longrightarrow \text{优化} \longrightarrow \text{泛化检验}}.$$

例如房价预测：面积和房龄是输入，成交价是目标；$\hat{y} = wx + b$ 是模型；$(y - \hat{y})^2$ 是单次平方损失；寻找合适的 w, b 是训练；在新房子上测试是泛化检验。

1.3　三种学习范式

范式	数据与反馈	典型问题
监督学习	每个输入有目标或标签	回归预测连续值；分类预测类别或类别概率
无监督学习	输入没有预先给定的标签	聚类寻找分组；PCA 寻找保留较多变化的方向
强化学习	行动改变环境，奖励可能延迟	游戏落子、机器人动作、长期决策

“没有标签”并不等于“没有目标”：K-means 仍优化簇内距离，PCA 仍优化投影后的方差。强化学习则要考虑当前行动怎样影响未来能看到的数据。

1.4 用课件案例读懂共同框架

- **DQN 与 AlphaGo：**游戏画面或棋盘是状态，动作会影响后续状态，胜负或得分给出反馈。AlphaGo 还把学到的策略、价值估计与树搜索组合。AlphaZero 借助可准确模拟的棋类规则进行大量自我对弈；现实道路或机器人通常没有这样低成本、完全可靠的试错环境。
- **AlphaFold：**输入蛋白质序列等信息，输出三维结构，利用已知结构训练。局部置信度与实验验证仍重要。
- **语言模型：**给前面的 token 预测下一个 token，文本自身提供训练目标，属于自监督学习的一种。语言流畅不保证事实正确。注意力机制使每个位置按当前需要汇总其他位置的信息；先记住这个作用，不必先推导 Transformer。
- **图像与视频生成：**根据文字条件生成可能的图像或视频，目标是学习一个条件分布，所以同一句描述可以产生不同结果。视频还需维持跨帧的一致性。
- **自动驾驶与机器人：**感知、预测、规划、控制形成闭环。平均准确率无法说明低概率高后果事件的风险；训练环境与真实环境之间可能发生分布偏移。

对任何新系统都可以追问：它看过什么、优化什么、怎样测试、在哪些场景可靠、失败会怎样。这比背产品名称更耐用。

1.5 常见分支怎样放进这张地图

深度学习让多层函数从数据中学习表示。CNN 复用局部卷积核，常见于图像；RNN 按时间更新隐藏状态，适合序列；Transformer 用注意力汇总不同位置的信息，广泛用于语言和多模态。两层纯线性变换仍能合并为一层，所以神经网络需要 ReLU 等非线性激活函数。模型更大并不自动解决数据质量、标注偏差或评估问题。

计算机视觉的输出形式各不相同：分类给整张图一个类别，检测给位置和类别，分割给每个像素类别。少数类重要时，只看准确率会误导。例如 100 张图仅有 2 张缺陷图，全部预测“正常”仍有 98% 准确率，却完全找不到缺陷。此时要看精确率 $TP/(TP+FP)$ 与召回率 $TP/(TP+FN)$，其中 TP 是正确报出的正例，FP 是误报，FN 是漏报。

自然语言处理处理文本的上下文与组合含义。检索增强生成（RAG）先从外部文档检索证据，再据此生成答案；若检索遗漏或证据本身错误，生成结果仍可能不可靠。**具身智能**把视觉、语言和动作放进反馈闭环；VLA（视觉—语言—动作）模型根据观察与指令输出动作，但真实机器人轨迹采集昂贵，仿真到现实还存在差距。这些系统都需要定义任务、数据、目标和失败检查。

第二章　读公式前的数学与 Python 预备

2.1　变量、函数、求和

x 是输入，y 是真实答案，$\hat{y}$ 读作 “预测的 y”。函数 $f(x)$ 是一个规则：给定 x，返回一个值。$\sum_{i=1}^{n} a_i$ 表示把 a_1 到 a_n 相加；$\frac{1}{n}\sum_i a_i$ 是平均。$\arg\min_w J(w)$ 表示 “使 J 最小的那个 w”，其结果是参数，不是最小值本身。

2.2　向量和矩阵只需先会看形状

把一个样本的 d 个特征写成 $x_i = (x_{i1}, \ldots, x_{id})^\top \in \mathbb{R}^d$。把 n 个样本按行堆起来：

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n\times d}, \qquad y = (y_1, \ldots, y_n)^\top \in \mathbb{R}^n.$$

因此**一行是一个样本，一列是一个特征**。$X^\top$ 是转置，即行列互换。$x^\top w = \sum_j x_j w_j$ 是点积，也就是线性模型的预测核心。$\|v\|_2^2 = \sum_i v_i^2$ 是平方和；$\|v\|_1 = \sum_i |v_i|$ 是绝对值之和。

假设 3 套房子的面积与房龄分别是 $(50, 2), (80, 8), (60, 5)$，则 X 形状为 3×2。如果权重 $w = (2, -1)^\top$，则 Xw 形状为 3×1，第一套的值为 $50 \cdot 2 + 2 \cdot (-1) = 98$。先检查形状，往往比先记矩阵公式更有效。

2.3　平均、方差和条件期望

均值是数据的平均水平。方差衡量围绕均值的平方波动。$\mathbb{E}[Y \mid X = x]$ 读作 “已知输入为 x 时，Y 的平均值”。它不同于 $\mathbb{E}[Y]$：后者不看输入，给所有人同一个预测。在平方损失下，条件期望是理论上最好的预测函数；在绝对损失下，对应的是条件中位数。

2.4　导数与梯度的直观理解

导数 dJ/dw 描述 w 小幅增加时 J 如何变化。负号方向通常让函数下降。多个参数时，各方向导数组成梯度 ∇J。梯度下降更新 $w \leftarrow w - \eta \nabla J(w)$，$\eta > 0$ 是学习率。学习率太大可能越过谷底，太小则走得慢。

2.5 Python 最小工具箱

在下面的代码中，列表如 `[1, 2, 3]` 保存一组值；`for` 重复执行；`sum` 求和；`len` 得到个数；`print` 显示结果。`import numpy as np` 引入 NumPy 并给它短名 `np`。`X.shape` 告诉你矩阵形状。`model.fit(X, y)` 用数据训练；`model.predict(X_new)` 预测新输入。两次调用的输入必须有相同的特征列顺序。

```
prices = [200, 220, 250]
mean_price = sum(prices) / len(prices)
for price in prices:
    print(price - mean_price)
```

先在 Python 解释器运行这段不依赖第三方库的代码。后文完整示例要安装 NumPy 与 scikit-learn；可在自己的虚拟环境中执行命令 `python -m pip install numpy scikit-learn`。

第三章　从数据到可检验的学习问题

3.1　数据不是现实本身

监督学习数据集记作 $D=\{(x_i,y_i)\}_{i=1}^{n}$。大写 X,Y 表示随机变量，小写 x_i,y_i 表示已经观察到的值。常用的独立同分布（i.i.d.）假设意味着样本由相同机制产生且彼此独立。它便于分析，却可能被时间趋势、同一家庭的关联样本、城市差异或采集方式破坏。

假如只采集市中心已成交房屋，再把模型用于远郊挂牌房屋，就同时改变了地理范围与价格定义。训练数据覆盖范围、缺失群体、测量误差和标签生成方式都需要记录。

3.2　训练误差与未来误差

单样本损失为 $\ell(y,f(x))$。训练集上的平均损失叫**经验风险**：

$$\widehat{R}_n(f)=\frac{1}{n}\sum_{i=1}^{n}\ell(y_i,f(x_i)).$$

对未来数据生成分布 P 的期望叫**总体风险**：

$$R(f)=\mathbb{E}_{(X,Y)\sim P}[\ell(Y,f(X))].$$

训练时能直接计算前者，真正希望降低的是后者。训练集上几乎零误差可以来自记住数据，而不是学到可迁移规律。

3.3　训练、验证、测试三种用途

训练集拟合 w,b 等**参数**；验证集选 K、多项式阶数、正则化强度等**超参数**；测试集只用于最终的一次评估。若反复看测试成绩并据此改模型，测试集就被“用来训练决策”了，分数会偏乐观。小数据可在训练部分使用交叉验证，但最后仍应保留独立测试证据。时间序列应按时间划分；同一个人的多条记录通常要按人分组，避免泄漏。

3.4　欠拟合、过拟合与偏差—方差

模型太简单，训练与测试都差，叫欠拟合；训练很好而新数据很差，叫过拟合。把同一算法在不同训练集上重复拟合：平均预测与真实条件均值的差叫偏差，预测随训练集变动的幅度叫方差。

在固定输入 x、平方损失和适当独立性条件下，

$$\mathbb{E}_{D,Y|x}\big[(Y-\hat{f}_D(x))^2\big] = \underbrace{\big(\mathbb{E}_D[\hat{f}_D(x)]-m(x)\big)^2}_{\text{偏差平方}} + \underbrace{\mathrm{Var}_D(\hat{f}_D(x))}_{\text{模型方差}} + \underbrace{\mathrm{Var}(Y\mid X=x)}_{\text{不可约噪声}},$$

$$m(x)=\mathbb{E}[Y\mid X=x].$$

增大 KNN 的 K、降低多项式阶数、加大正则化强度常会降低方差、增加偏差。方向通常如此，具体最优值仍要靠验证集确定。

> **记住这一点：**预测关系与因果关系不同。面积系数为正，只说明在模型与样本条件下面积和价格有关；“把某套房子的面积增加一平方米会使售价增加多少”需要更强的因果设计。

第四章　回归：连续数值预测

4.1　什么是回归

回归预测一个连续数值：房价、耗电量、温度或交付时长。分类预测离散类别或类别概率，如“是否垃圾邮件”。预测是估计未知数值；决策还要结合价格、成本、风险与约束。即便模型预测某房售价为 200 万元，也不能单凭此数决定是否买入。

4.2　回归函数与不可约噪声

定义 $m(x) = \mathbb{E}[Y \mid X = x]$ 和 $\varepsilon = Y - m(X)$，于是

$$Y = m(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0.$$

这个写法不要求 m 线性。即使完全知道 m，同样的输入仍可能出现不同输出：交易谈判、测量误差、未记录特征都会引起波动。因此模型误差无法无限降为零。

4.3　损失与评价指标

设误差 $e_i = y_i - \hat{y}_i$。常用指标为

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|, \quad \mathrm{MSE} = \frac{1}{n}\sum_i e_i^2, \quad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \quad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}.$$

MAE 与 RMSE 的单位和 y 一样，MSE 的单位是 y 单位的平方。大误差在平方损失中权重更高。$R^2 = 0$ 表示在*同一评估数据*上与用该数据的均值作常数预测一样，$R^2 < 0$ 表示更差；若分母为零，通常不能按此式解释。MAPE 对真值除法，$y_i = 0$ 时无定义，接近零时会异常放大。

手算： $y = (3, 5, 2)$，$\hat{y} = (2, 5, 4)$，绝对误差是 $(1, 0, 2)$，所以 $\mathrm{MAE} = 1$；平方误差和为 5，所以 $\mathrm{RMSE} = \sqrt{5/3} \approx 1.291$。$\bar{y} = 10/3$，总平方和为 $14/3$，因而 $R^2 = 1 - 5/(14/3) = -1/14 \approx -0.071$。这里的负分说明这三个预测甚至不如常数基线。

选择指标要依据任务代价：预测高估和低估若有不同后果，单一对称指标可能不足。报告指标时还要给样本量、划分方式与目标单位。

第五章　K 近邻回归：从“像谁”开始

5.1　算法与手算

对新输入 x，计算它到每个训练输入 x_i 的距离，选最近的 K 个，平均它们的 y_i：

$$\hat{f}_K(x) = \frac{1}{K} \sum_{i \in N_K(x)} y_i.$$

例如有下面 5 个“面积、离中心距离、月租”样本。查询 $(70, 2.5)$，若暂不标准化并用欧氏距离，最近三点是第 2、3、1 行，预测 $(2500 + 2800 + 3000)/3 \approx 2766.7$ 元。这个数只是演示距离计算，不是实际租金估计。

样本	面积（平方米）	距离（公里）	月租（元）	到查询点的距离
1	50	1	3000	20.06
2	80	3	2500	10.01
3	60	2	2800	10.01
4	100	5	2000	30.10
5	40	1.5	3200	30.02

欧氏距离是 $d(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$；曼哈顿距离是 $\sum_j |x_j - z_j|$。距离定义和 K 都是要选择的超参数。小 K 易受单个异常样本影响，大 K 易把局部差异平均掉。$K = n$ 时预测恒为训练目标均值。

5.2　为什么必须关注特征尺度

面积可能以几十平方米变化，距离只在几公里变化。直接平方相加，面积往往主导“近”。标准化每列为

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

其中 μ_j, s_j **只从训练集计算**；验证、测试和新数据都复用这些数。若在划分前对全部数据标准化，测试数据的信息已进入训练流程，属于数据泄漏。若某列标准差为零，应移除或使用可处理常数列的预处理器。

KNN 几乎不需训练，但预测时需查找历史样本；它在训练范围外的外推能力很弱。高维时许多点的距离会变得难区分，需要特征选择、降维或另一类模型。

第六章　线性回归：从直线到矩阵

6.1　模型、截距、残差

一个特征时 $\hat{y} = wx + b$；多个特征时 $\hat{y} = b + \sum_{j=1}^{d} w_j x_j$。$w_j$ 表示在模型中保持其他特征不变时，该特征增加一个单位，预测值怎样变化；它本身不是因果效应。残差是 $e_i = y_i - \hat{y}_i$。把 X 左边加一列全 1 得到 $\widetilde{X}$，把 b 放进参数向量 β，就有 $\hat{y} = \widetilde{X}\beta$。

6.2　最小二乘及其解

令目标函数

$$J(\beta) = \frac{1}{2n}\|\widetilde{X}\beta - y\|_2^2.$$

前面的 $1/2$ 让求导后的 2 抵消，不改变最优解。梯度为

$$\nabla J(\beta) = \frac{1}{n}\widetilde{X}^\top(\widetilde{X}\beta - y).$$

令梯度为零，得到**正规方程**

$$\widetilde{X}^\top \widetilde{X}\hat{\beta} = \widetilde{X}^\top y.$$

若 $\widetilde{X}$ 列满秩，$\hat{\beta} = (\widetilde{X}^\top\widetilde{X})^{-1}\widetilde{X}^\top y$。目标函数是凸二次函数，所以满足正规方程的解是全局最优；列不满秩时可存在多个系数解，但最小二乘预测可由伪逆给出。

务必核算的三点例子：课件给了 $(1,3),(2,5),(4,4)$，但课件该页印出的拟合直线和残差数值与这三个点不一致。直接用均值法：$\bar{x} = 7/3$，$\bar{y} = 4$，

$$\hat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{1}{14/3} = \frac{3}{14}, \qquad \hat{b} = \bar{y} - \hat{w}\bar{x} = \frac{7}{2}.$$

所以正确的直线是 $\hat{y} = \frac{7}{2} + \frac{3}{14}x$。三点预测为 $26/7, 55/14, 61/14$，残差为 $-5/7, 15/14, -5/14$；残差之和为 0。你可以把三项平方相加，再尝试轻微改变斜率，看平方和是否增大。

6.3　几何与数值计算

$\hat{y} = \widetilde{X}\hat{\beta}$ 是把 y 投影到 $\widetilde{X}$ 的列空间；残差向量与该列空间正交，即 $\widetilde{X}^\top(y - \hat{y}) = 0$。几何上这解释了正规方程。显式计算 $(X^\top X)^{-1}$ 在数值上可能放大误差，尤其是特征高度相关或尺度差异很大时。实际软件通常使用 QR、SVD 或其他稳定求解方法；做手算时写逆矩阵公式，写程序时优先使用稳定求解器。标准化能改善尺度问题，但不能自动消除真正的共线性。

6.4 统计假设与解释边界

平方损失最小化本身不要求噪声正态。若另加 $Y = X\beta + \varepsilon$、$\mathbb{E}[\varepsilon \mid X] = 0$、各样本误差不相关且同方差等条件，便可讨论 OLS 的无偏性与 Gauss–Markov 定理；进一步假设正态通常方便建立精确的有限样本检验。异方差或相关误差会影响常规标准误，遗漏变量可能让系数解释失效。对零基础读者，先会检查残差、预测区间与数据来源，再学习显著性检验。

第七章　梯度下降：不会一步算出时怎样找答案

对 $J(\beta)=\|X\beta-y\|_2^2/(2n)$，梯度下降是

$$\beta^{(t+1)}=\beta^{(t)}-\eta\frac{1}{n}X^\top(X\beta^{(t)}-y).$$

每一步先用当前参数预测，再算残差，再把梯度反方向加到参数上。批量梯度下降每步用全部样本；随机梯度下降每步用一个样本；小批量版本介于两者之间。标准化常让不同方向的坡度更接近，便于设置学习率。对这个二次目标，若 η 太大可能发散；若太小会收敛得很慢。常见的检查方式是记录每轮损失，确认总体下降，并同时检查验证集表现。

极简演练：固定截距为零，只有一个样本 $(x,y)=(1,2)$，$J(w)=\frac{1}{2}(w-2)^2$，梯度 $w-2$。从 $w_0=0$，取 $\eta=0.5$，得到 $w_1=1$、$w_2=1.5$、$w_3=1.75$，逐步靠近 2。这个例子让“负梯度方向”有了可算的含义。

第八章　复杂模型、正则化与概率解释

8.1　多项式仍可用线性回归训练

把原输入 x 变为 $\phi(x) = (1, x, x^2, \ldots, x^k)$，模型 $\hat{y} = \beta^\top \phi(x)$ 对 x 是曲线，但对参数 β 仍线性，因此最小二乘推导不变。阶数升高会增加表达能力，也可能放大噪声和数值不稳定。强相关的高次幂使矩阵接近奇异；可考虑标准化、较低阶数、合适基函数和正则化。

8.2　Ridge、Lasso、Elastic Net

为统一推导，以下把不惩罚截距的特征矩阵记为 X，目标值已按需要中心化，定义

$$J_{\text{ridge}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2, \qquad J_{\text{lasso}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1.$$

Ridge 的方程是 $(X^\top X + \lambda I)\hat{w} = X^\top y$。$\lambda > 0$ 时矩阵正定，能稳定缩小系数，通常不会把某个系数精确压成零。Lasso 使用绝对值惩罚，可以让部分系数恰为零；没有类似 Ridge 的通用矩阵闭式解。Elastic Net 同时包含两种惩罚，适合需要稀疏性且特征成组相关的场景。

方法	系数效果	相关特征	常见用途
Ridge	整体缩小，通常不精确归零	常共同保留	多数特征有用，系数不稳
Lasso	部分系数精确归零	可能择其一，选择不稳	希望得到较少特征
Elastic Net	缩小且可归零	更倾向共同保留	特征分组相关

量纲很重要：正则化前通常要标准化特征，否则同样的惩罚对不同单位不公平。截距通常不罚。λ 的数值依赖前面损失是否除以 n、是否写 $1/2$；课件、PDF 和软件包中的 `alpha` 不能看到数字就直接等同。应先对齐目标函数，再比较数值。

8.3　为什么高斯假设会导出平方损失

如果假设各样本在给定 x_i 下独立，且 $Y_i \mid x_i \sim \mathcal{N}(x_i^\top w, \sigma^2)$，则负对数似然中与 w 有关的部分是

$$-\log L(w) = \frac{1}{2\sigma^2}\sum_i (y_i - x_i^\top w)^2 + \text{与} w \text{ 无关的常数}.$$

所以最大似然估计 w 与最小二乘同解。这个推导是一种解释，不是使用最小二乘必须满足的先决条件。若噪声重尾，平方误差可能过度重视离群点，需要检查残差并考虑其他损失。

8.4 进阶选读：软阈值与 ISTA

对一个数 z，软阈值算子是

$$S_\tau(z) = \operatorname{sign}(z)\max(|z| - \tau, 0).$$

例如 $S_2(5) = 3$，$S_2(1) = 0$，$S_2(-4) = -2$。若 $g(w) = \frac{1}{2}\|Xw - y\|_2^2$，Lasso 可用迭代软阈值算法（ISTA）：

$$u^{(t)} = w^{(t)} - \eta X^\top (Xw^{(t)} - y), \qquad w_j^{(t+1)} = S_{\eta\lambda}(u_j^{(t)}).$$

先沿光滑的平方损失下降，再让每个坐标向零收缩。步长可取 $0 < \eta \le 1/\|X\|_2^2$，这里 $\|X\|_2$ 是最大奇异值。实践中还要规定停止条件，例如相邻两步参数变化足够小；若目标值反而升高，先检查步长和公式约定。

第九章　完整的回归工作流

1. **定义问题：**要预测什么，何时预测，预测时确实可取得哪些特征，误差代价如何。
2. **核对数据：**单位、缺失、异常、样本覆盖范围和标签生成过程。
3. **先划分：**保留独立测试集；有时间或群组结构时相应划分。
4. **建立基线：**先试“总预测训练均值”，再试简单线性模型，最后才增加复杂度。
5. **预处理与建模：**仅在训练折拟合标准化或填补器；用流水线连接预处理与模型。
6. **验证选择：**比较 KNN、线性回归、Ridge 等模型及其超参数，关注方差和样本量。
7. **最终测试：**报告 MAE、RMSE、R^2 和划分方式；检查残差是否随预测值、时间或群组系统变化。
8. **部署监控：**记录输入分布、缺失率与误差；出现分布漂移时重新评估。

两类常见失败：数据泄漏指特征包含预测时拿不到的信息，例如用按成交价计算的中介费预测成交价；分布漂移指训练环境和实际使用环境不同，例如仅用市区房屋训练却预测远郊。两者都可能让离线分数失去实际意义。

第十章　跟着敲两段 Python

10.1　不用第三方包，手算误差

保存为 `metrics.py` 后运行 `python metrics.py`；也可逐行粘贴到 Python 解释器。代码中每一步都对应第 4 章公式。

```python
y_true = [3, 5, 2]
y_pred = [2, 5, 4]
n = len(y_true)
errors = [a - b for a, b in zip(y_true, y_pred)]
mae = sum(abs(e) for e in errors) / n
mse = sum(e * e for e in errors) / n
rmse = mse ** 0.5
y_mean = sum(y_true) / n
tss = sum((y - y_mean) ** 2 for y in y_true)
r2 = 1 - sum(e * e for e in errors) / tss
print(mae, mse, rmse, r2)
# Expected: 1.0, 1.666..., 1.29099..., -0.071428...
```

注意 `zip` 会把两组列表按位置配对，`** 0.5` 是开平方。若两组长度不等，`zip` 会静默截短，正式代码应先检查长度相等。

10.2　可重复的模拟数据练习

以下代码生成一批模拟房价，只用来观察完整流程；输出数值不代表真实房地产结论。安装依赖后在同一脚本中运行。`random_state` 固定随机种子，便于复现；`Pipeline` 让标准化只在各训练折学习。R^2 是评估指标，不是每一步的训练损失。

```python
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

rng = np.random.default_rng(42)
area = rng.uniform(45, 140, size=120)
```

```
age = rng.uniform(0, 25, size=120)
distance = rng.uniform(1, 15, size=120)
X = np.column_stack([area, age, distance])
noise = rng.normal(0, 12, size=120)
y = 30 + 2.8 * area - 1.5 * age - 4.0 * distance + noise

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
models = {
    "mean": None,
    "linear": LinearRegression(),
    "ridge": make_pipeline(StandardScaler(), Ridge(alpha=10.0)),
    "knn": make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=5)),
}
for name, model in models.items():
    if model is None:
        pred = np.full(len(y_test), y_train.mean())
    else:
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    print(name, "MAE", round(mae, 2), "RMSE", round(rmse, 2), "R2", round(r2, 3))

# Only train folds are used during parameter selection.
search = GridSearchCV(
    make_pipeline(StandardScaler(), KNeighborsRegressor()),
    {"kneighborsregressor__n_neighbors": [3, 5, 9, 15]},
    scoring="neg_mean_absolute_error", cv=5
)
search.fit(X_train, y_train)
print("selected K:", search.best_params_)
print("test MAE:", mean_absolute_error(y_test, search.predict(X_test)))
```

把 `noise` 的标准差从 12 改成 40，再观察各方法；或者把真实关系中的 `area` 改为平方项，看看线性模型何时开始欠拟合。**不要**根据最后一行测试分数反复挑 K，那会把测试集变成验证集。这里比较多个模型的测试输出仅为教学演示；正式评估应在训练数据内部完成所有选择后，只评估一次最终模型。

第十一章　自测题与参考答案

题目

1. 给定任务“预测未来一天用电量”，写出 T, P, E，并说明输出是回归还是分类。
2. X 有 100 行、3 列，w 有 3 个元素。Xw 的形状是什么？若加截距应怎样处理？
3. 训练 MAE 为 1、测试 MAE 为 20，可能发生了什么？至少说出两个检查方向。
4. 为什么计算 KNN 距离之前要标准化？标准化参数应从哪部分数据计算？
5. 在 $(1,3),(2,5),(4,4)$ 上，最小二乘直线的斜率与截距是多少？
6. 如果 $R^2 = -0.2$，这在当前评估集上意味着什么？
7. Ridge 和 Lasso 哪个更容易得到精确的零系数？为什么？
8. 从 $w_0 = 0$ 出发，对 $J(w) = \frac{1}{2}(w-2)^2$ 取 $\eta = 0.5$，算两步梯度下降。
9. 用“未来才知道的维修费”预测当前房价有什么问题？

答案与理由

1. T 是预测下一天的连续用电量，P 可用 MAE（单位千瓦时），E 是历史用电、天气和日历等已知记录；这是回归。
2. Xw 有 100 个预测值，形状通常写作 100×1 或 NumPy 的 `(100,)`。加截距可用 $Xw + b$，也可给 X 加全 1 列并把 b 放入参数。
3. 可能过拟合、泄漏、划分方式不当或分布变化。先核对训练/测试划分、预处理是否在训练集内拟合，再检查两边目标值与特征范围。
4. 不同量纲会不公平地支配距离；均值与标准差只从训练数据计算，验证/测试复用。
5. $\hat{w} = 3/14$，$\hat{b} = 7/2$。可代入三点求残差并确认残差和为零。
6. 当前模型的平方误差和比“总预测该评估集均值”的基线高 20%。不能单凭这个数字判断原因。
7. Lasso。绝对值惩罚的软阈值会把小系数精确压到零；Ridge 常把系数连续缩小。
8. $w_1 = 1$，$w_2 = 1.5$；目标值也从 2 降到 0.5 再降到 0.125。
9. 预测时拿不到维修费，属于时间上的数据泄漏；离线分数可能虚高。

第十二章　一页公式索引与术语表

概念	公式或含义
经验风险	$\widehat{R}_n(f) = n^{-1}\sum_i \ell(y_i, f(x_i))$
平方损失下的最佳回归函数	$m(x) = \mathbb{E}[Y \mid X = x]$
线性模型	$\hat{y} = Xw + b$，其中矩阵按行放样本
最小二乘	$\min_w \|Xw - y\|_2^2$
正规方程	$X^\top X\hat{w} = X^\top y$；含截距时使用增广矩阵
梯度下降	$w \leftarrow w - \eta\nabla J(w)$
Ridge	平方损失加 $\lambda\|w\|_2^2$；系数收缩
Lasso	平方损失加 $\lambda\|w\|_1$；可产生零系数
MAE / RMSE	平均绝对误差 / 均方根误差，均与目标同单位
R^2	$1 - \mathrm{RSS}/\mathrm{TSS}$，可为负
过拟合	训练集误差低而新数据误差高
数据泄漏	预测时不可获得的信息进入训练特征或预处理
分布漂移	实际输入或目标机制偏离训练数据

第十三章　资料对应与继续学习

13.1　这份讲义怎样对应原始资料

- **《机器学习简介 · 第一讲 (1).pptx》：**第 1 章对应案例、TPE 定义与学习范式；第 3 章对应数据、经验风险、泛化及数据划分；第 10 章对应系统工作流。其大模型、视觉、自动驾驶、机器人部分在第 1 章作概念地图，避免在回归入门前展开过多细节。
- **《机器学习导论 · 第 2 讲：回归.pptx》：**第 4–10 章对应回归目标、KNN、线性回归、梯度下降、多项式、正则化及工作流。第 6 章重新核算了该课件三点最小二乘例子。
- **《2-regression.pdf》：**第 3–4 章对应条件期望、训练/测试风险及偏差—方差；第 6–9 章对应 OLS、梯度下降、Ridge/Lasso 和 ISTA 的推导。本文将 ISTA 标记为进阶选读，便于先建立直觉。

13.2　下一步学什么

掌握这两讲后，下一站通常是分类：先理解逻辑回归输出概率，再学交叉熵与精确率、召回率；随后学习交叉验证、决策树、聚类和 PCA。复习时反复使用本讲的共同问题：*数据从哪里来？模型允许什么规律？优化什么？如何验证？在哪些情境会失败？*

13.3　软件资料

本讲 Python 示例按 scikit-learn 官方 API 编写；安装包和版本会变化，运行前可查官方文档：https://scikit-learn.org/stable/。`train_test_split`、`Pipeline` 与 `GridSearchCV` 的详细参数见该站 API 文档。若在虚拟环境中出现导入错误，先检查当前 `python` 与安装包是否属于同一个环境。